

COP3410 Florida Atlantic University Assignment 4

100 points

1. **Given an unsorted list of n values, what is the time-complexity to find the k th smallest value in the worst case? What would be the complexity if the list were sorted?** An unsorted list would have to search the whole list to find the k th smallest value, which is $O(n)$. While the sorted list would be the k th element of that list, $O(1)$.
2. **Modify the binary search algorithm to find the position of the first occurrence of a value that can occur multiple times in the ordered list. Verify your algorithm is still $O(\log n)$.** The algorithm just forces its worst case instead of stopping when the first instance is found, which is still $O(\log n)$.

```
"""
Coded by: Leyon Anderson
3/28/2026
COP3410 Assignment #4

Goal is to modify the binary search algorithm to find the position
of the first occurrence
of a value that can occur multiple times in the ordered list.
"""

def fOccurence(lst, target):
    """
    A binary search algorithm to find the position of the first
    occurrence of a value.

    :param list lst: List of integers.
    :param int target: The target to find.
    """
    low = 0
    high = len(lst) - 1
    result = -1 # Default if not found

    while low <= high:
        mid = (low + high) // 2

        if lst[mid] == target:
```

```

        result = mid      # Track location if match found
        high = mid - 1    # Keep looking to the left (to find
earlier occurnaces)
        elif lst[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return result

if __name__ == "__main__":
    test_case = [1, 2, 2, 3, 4, 5, 6, 6]    # Test case list
    target = 6                             # Test case target

    result = fOccurence(test_case, target)

    print(f'{f"{target}" was first found at index {result}.' if
result != -1 else f'{target} was not found.'}')

```

3. **Design and implement a function to find all negative values within a given list. Your function should output and return a new list containing the negative values. When does the worst case occur and what is the run time for that case?** Assuming a sorted list, the worst case would occur if the entire list contains negative values with a run time of $O(n)$ the same as an unsorted list. My implementation is always $O(n)$ not needing a sorted list but always traversing the whole list.

```

"""
Coded by: Leyon Anderson
3/28/2026
COP3410 Assignment #4

Goal is to design and implement a function to find all negative
values within a
given list. The function should output and return a new list
containing the
negative values.
"""

```

```
def fNeg(lst):
    '''
    Find all negative values within a list of integers.

    :param list lst: List of integers.
    '''
    return [x for x in lst if x < 0]

if __name__ == "__main__":
    print(fNeg([1, -1, -2, 3, 4])) # Function call contains the list
    to test.
```

4. Given the following list of keys (80, 7, 24, 16, 43, 91, 35, 2, 19, 72), show the contents of the array after each iteration of the outer loop for the indicated algorithm when sorting in ascending order.

a. bubble sort

(80, 7, 24, 16, 43, 91, 35, 2, 19, 72)
 (7, 24, 16, 43, 80, 35, 2, 19, 72, 91)
 (7, 16, 24, 43, 35, 2, 19, 72, 80, 91)
 (7, 16, 24, 35, 2, 19, 43, 72, 80, 91)
 (7, 16, 24, 2, 19, 35, 43, 72, 80, 91)
 (7, 16, 2, 19, 24, 35, 43, 72, 80, 91)
 (7, 2, 16, 19, 24, 35, 43, 72, 80, 91)
 (2, 7, 16, 19, 24, 35, 43, 72, 80, 91)
 (2, 7, 16, 19, 24, 35, 43, 72, 80, 91)
 (2, 7, 16, 19, 24, 35, 43, 72, 80, 91)
 (2, 7, 16, 19, 24, 35, 43, 72, 80, 91)

b. selection sort

(80, 7, 24, 16, 43, 91, 35, 2, 19, 72)
 (2, 7, 24, 16, 43, 91, 35, 80, 19, 72)
 (2, 7, 24, 16, 43, 91, 35, 80, 19, 72)
 (2, 7, 16, 24, 43, 91, 35, 80, 19, 72)
 (2, 7, 16, 19, 43, 91, 35, 80, 24, 72)
 (2, 7, 16, 19, 24, 91, 35, 80, 43, 72)
 (2, 7, 16, 19, 24, 35, 91, 80, 43, 72)
 (2, 7, 16, 19, 24, 35, 43, 80, 91, 72)
 (2, 7, 16, 19, 24, 35, 43, 72, 91, 80)
 (2, 7, 16, 19, 24, 35, 43, 72, 80, 91)

(2, 7, 16, 19, 24, 35, 43, 72, 80, 91)

c. insertion sort

(80, 7, 24, 16, 43, 91, 35, 2, 19, 72)

(80, 7, 24, 16, 43, 91, 35, 2, 19, 72)

(7, 80, 24, 16, 43, 91, 35, 2, 19, 72)

(7, 24, 80, 16, 43, 91, 35, 2, 19, 72)

(7, 16, 24, 80, 43, 91, 35, 2, 19, 72)

(7, 16, 24, 43, 80, 91, 35, 2, 19, 72)

(7, 16, 24, 43, 80, 91, 35, 2, 19, 72)

(7, 16, 24, 35, 43, 80, 91, 2, 19, 72)

(2, 7, 16, 24, 35, 43, 80, 91, 19, 72)

(2, 7, 16, 19, 24, 35, 43, 80, 91, 72)

(2, 7, 16, 19, 24, 35, 43, 72, 80, 91)

5. **Evaluate the insertion sort algorithm to determine the best-case and the worst-case time complexities.** Best case the list is already sorted so $O(n)$. The worst case is the list is in reverse order so the time complexity is $O(n^2)$
6. **Use the built-in sum function combined with Python's comprehension syntax to compute the sum of all numbers in an $n \times n$ data set, represented as a list of lists.**

```
"""
Coded by: Leyon Anderson
3/28/2026
COP3410 Assignment #4

Uses the built-in sum function combined with Python's comprehension
syntax to
compute the sum of all numbers in an N*N data set, represented as a
list of lists.
"""

nxn = [[1, 2, 3],[1, 2, 3],[1, 2, 3]] # This is the list to test.

allSum = sum([sum(nxn[x]) for x in range(len(nxn))])
print(allSum)
```